Royale Cars — Roteiro do vídeo

Oct 22, 2026 · @Caio Andrade Campos

Um vídeo de cerca de 12 minutos, gravado em 10 blocos curtos (tela e voz, sem câmera) no notebook do Caio e juntado no editor. São 7 minutos de sistema funcionando e 4 de código, contando a história de um aluguel que passa pelas 3 plataformas.

O vídeo em 10 blocos

Cada um grava os seus blocos; o total dá cerca de 12 min 30 s, com folga para os 15. A Parte 1 segue um aluguel do começo ao fim: a web cadastra a cliente, o celular aluga com assinatura, o desktop vê o contrato e devolve, e a web mostra o dinheiro entrando.

#	Bloco	Quem	Tempo	O que aparece
1	Abertura: o problema e a solução	Caio	1:00	Desktop no Dashboard e o diagrama de arquitetura da documentação
2	Web: cadastro da cliente e frota	Gabriel	1:30	Web: login, Dashboard, Clientes, Frota
3	Celular: aluguel com assinatura	Evaldo	2:00	App: login, painel, Nova Locação, assinatura, contrato PDF
4	Desktop: contrato, devolução e oficina	Caio	2:00	Desktop: Frota, Locações, Contrato, Devolução, Manutenção
5	Web: financeiro, relatórios e administração	Gabriel	1:15	Web: Financeiro, Relatórios, Administração
6	Código: o banco de dados	Caio	1:15	VS Code: `banco/02_complemento.sql` e `01_royale_cars_supabase.sql`
7	Código: a web	Gabriel	1:00	VS Code: `web/includes/auth.php` e `funcoes.php`
8	Código: o celular	Evaldo	1:00	VS Code: `locacao_controller.dart` e `supabase_service.dart`
9	Código: o desktop	Caio	1:00	VS Code: `preload.js`, `ipc/locacoes.js`, `ipc/contratos.js`
10	Encerramento	Evaldo	0:30	Os 3 sistemas abertos lado a lado

Tempo de fala de cada um: Caio 5:15, Gabriel 3:45, Evaldo 3:30.

Antes de gravar

Gravem os blocos **na ordem de 1 a 10 e no mesmo dia**: cada bloco usa o que o anterior deixou no banco (a cliente do bloco 2 é a que aluga no bloco 3, e assim por diante).

Dados

- [ ] Rodar `banco/07_restaurar_demo.sql` no Supabase na manhã da gravação (as datas contam a partir do dia em que ele roda).
- [ ] Ter à mão a cliente nova do bloco 2 (documentos válidos, já conferidos): **Marina Alves**, CPF 614.775.052-61, CNH 42758083250, validade da CNH 10/05/2031, telefone (19) 99123-4567, e-mail marina.alves@gmail.com.

- [ ] Ter as senhas das contas de demonstração (Gabriel = chefe, Lucas = vendedor). Não escrevam senha neste documento.

Notebook

- [ ] Ligado na tomada, tela em 1920 × 1080.
- [ ] Notificações desligadas (Windows: "Não perturbe") e WhatsApp, Discord e e-mail fechados.
- [ ] Desktop (`npm start`), web (XAMPP) e celular (`flutter run -d chrome`) já abertos e com login feito antes de cada bloco.
- [ ] Celular no Chrome com cara de celular: F12, depois Ctrl + Shift + M, aparelho "Pixel 7"; mova o DevTools para uma janela separada para ele não aparecer no vídeo.
- [ ] VS Code com fonte grande (Ctrl + = até uns 18), minimapa desligado e só os arquivos do bloco abertos.

Gravador

- [ ] OBS Studio (grátis): Captura de janela, áudio só do microfone, 1920 × 1080, 30 quadros, saída em MP4. Sem OBS: Win + Alt + R (Xbox Game Bar) grava a janela da frente.
- [ ] Fone com microfone, lugar sem barulho e um teste de 10 segundos antes de cada pessoa gravar.

Parte 1 — O sistema funcionando

As falas são um guia: digam com as palavras de vocês, sem ler. Cada fala tem cerca de 130 palavras por minuto, que é um ritmo calmo.

Bloco 1 — Abertura (Caio, 1:00)

Na tela

1. Desktop aberto no Dashboard.
2. Aos 30 segundos, mostrar a figura de arquitetura da documentação (Figura 1) em tela cheia.

Fala

> Olá! Nós somos o Caio, o Gabriel e o Evaldo, e este é o Royale Cars, nosso TCC do Cotil. O Royale Cars é um sistema de gestão para locadoras de carros: cadastro de clientes e da frota, aluguel com contrato assinado, devolução, manutenção e financeiro.
>
> O desafio do projeto foi fazer o mesmo sistema funcionar em três plataformas: na web, feita em PHP pelo Gabriel; no computador, feita em Electron por mim; e no celular, feita em Flutter pelo Evaldo. As três usam um único banco de dados na nuvem, o Supabase. Então o que acontece numa plataforma aparece na hora nas outras. Para mostrar isso, vamos acompanhar um aluguel do começo ao fim, passando pelas três.

Bloco 2 — Web: a cliente chega na locadora (Gabriel, 1:30)

Na tela

1. Tela de login da web. Entrar como chefe.
2. No Dashboard, passar o mouse pelos 4 cartões e pelos Alertas.
3. Clientes → Cadastrar cliente. Preencher a Marina, mas digitar primeiro o CPF errado `111.111.111-11` e tentar salvar, para aparecer o aviso.
4. Corrigir para 614.775.052-61 e salvar.
5. Frota: mostrar os cartões com Disponível, Alugado e Em manutenção. Parar no Audi A5.

Fala

> Eu sou o Gabriel e vou mostrar a versão web. Ao entrar, o Dashboard mostra o resumo do dia: o faturamento do mês, os carros alugados, os aluguéis atrasados e os alertas, como uma CNH perto de vencer.
>
> Chegou uma cliente nova, a Marina. Se eu digito um CPF inválido, o sistema não deixa salvar: os dígitos verificadores são conferidos, e não só aqui na tela, mas também no banco de dados. Assim nenhuma das três plataformas consegue gravar um CPF errado. Com o CPF certo, ela está cadastrada.
>
> Na Frota vemos cada veículo e a situação dele: disponível, alugado ou na oficina. O Audi A5 está disponível, e agora o Evaldo vai alugar esse carro para a Marina pelo celular.

Bloco 3 — Celular: o aluguel com assinatura (Evaldo, 2:00)

Na tela

1. Login no app como o Lucas (vendedor). Mostrar que o menu tem só o que ele pode usar.
2. Painel: o resumo e os alertas.
3. Nova Locação → cliente Marina Alves → veículo Audi A5. A quilometragem aparece sozinha.
4. Devolução prevista: daqui a 3 dias. O valor previsto muda na hora.
5. Assinar contrato e confirmar → assinar devagar com o mouse → Confirmar assinatura.
6. Na janela "Aluguel registrado", clicar em Ver contrato (PDF) e rolar o contrato até a assinatura.

Fala

> Eu sou o Evaldo e vou mostrar o aplicativo. O celular é a ferramenta do atendimento, usada no balcão, junto com o cliente. Entrei como o Lucas, que é vendedor: o menu dele mostra só o que ele tem permissão para usar. O financeiro e a administração nem aparecem.
>
> Vou fazer o aluguel da Marina. Escolho a cliente e o carro, e a quilometragem atual já vem preenchida. Coloco a devolução para daqui a três dias, e o app calcula o valor previsto pela diária do carro.
>
> Agora a parte mais importante: a cliente assina o contrato aqui na tela, com o dedo. Confirmando, o aluguel é registrado, a assinatura fica guardada na nuvem e o carro passa a aparecer como alugado em todas as plataformas. Já dá para abrir o contrato em PDF, com a assinatura dela, e enviar ou imprimir. Agora o Caio vai continuar esse aluguel no computador.

Bloco 4 — Desktop: contrato, devolução e oficina (Caio, 2:00)

Na tela

1. Frota: o Audi A5 agora está Alugado, sem ninguém ter mexido no desktop.
2. Locações → Em andamento → ícone do contrato na linha da Marina. Mostrar a assinatura vinda do celular e o botão Salvar PDF.
3. Na linha da Fernanda Lima (Honda Civic, atrasada 2 dias) → Devolver.
4. Preencher a quilometragem de volta, uma taxa extra de R$ 80,00 com o motivo "Tanque na reserva" e marcar que voltou com defeito: "Barulho no freio". Mostrar o resumo com o atraso e confirmar.
5. Manutenção → Na oficina: o Civic aparece com "veio da devolução". Clicar em Liberar, serviço "Troca das pastilhas de freio", custo R$ 450,00, confirmar.

Fala

> No desktop, o Audi que o Evaldo alugou já aparece como alugado, sem eu atualizar nada. Em Locações, abro o contrato da Marina: é o mesmo contrato, com a assinatura que ela fez no celular, e posso salvar em PDF ou imprimir.
>
> Agora outra situação comum: a Fernanda devolveu o Civic com dois dias de atraso, o tanque na reserva e um barulho no freio. Informo a quilometragem, a taxa com o motivo e marco o defeito. O sistema calcula sozinho as diárias, o atraso e o total. Como o carro voltou com defeito, ele vai direto para a oficina e fica bloqueado para aluguel.
>
> Na tela de Manutenção ele aparece, ligado a essa devolução. Quando a oficina termina, eu libero: informo o serviço e o custo. O carro volta para a frota, e esse custo entra no financeiro, que o Gabriel vai mostrar agora.

Bloco 5 — Web: o dinheiro e a equipe (Gabriel, 1:15)

Na tela

1. Financeiro: os cartões do mês e o gráfico de 12 meses.
2. Relatórios → tipo Locações → Exportar CSV.
3. Administração → Funcionários → abrir o Lucas e mostrar as permissões marcadas.
4. Administração → Tentativas de login.

Fala

> De volta à web. No Financeiro, a devolução que o Caio acabou de fazer já entrou no faturamento do mês, e o custo da oficina entrou em manutenção. O gráfico mostra os últimos doze meses. Em Relatórios, dá para exportar os dados para uma planilha.
>
> Na Administração, que só o chefe acessa, ele cadastra a equipe e escolhe, uma por uma, as treze permissões de cada funcionário. Foi por isso que o Lucas viu um menu menor no celular. E aqui ficam as tentativas de login: depois de cinco senhas erradas, o e-mail fica bloqueado por quinze minutos.

Parte 2 — Por dentro do código

O fio condutor é o mesmo aluguel da Parte 1, agora visto por dentro. Em cada trecho, selecionem as linhas com o mouse enquanto falam. Embaixo de cada fala está o "porquê" da escolha: é o que vale estudar, porque é o que o professor vai perguntar.

Bloco 6 — O banco de dados (Caio, 1:15)

Abrir no VS Code

1. `banco/02_complemento.sql`, função `registrar_aluguel` (linha 337). Selecionar a linha 351 (`exigir_permissao`), a 369 (CNH vencida), a 376 (`for update`) e a 398 (`update veiculos set status = 'Alugado'`).
2. `banco/01_royale_cars_supabase.sql`, linha 205: o índice `uk_locacao_ativa_por_veiculo`.
3. Mesmo arquivo, linha 269 (`tem_permissao`) e linha 1066 (a regra `locacoes_ver`).

Fala

> O coração do sistema está no banco. Quando qualquer plataforma registra um aluguel, ela chama esta função, a `registrar_aluguel`. Primeiro ela confere a permissão de quem está pedindo. Depois confere as regras: a CNH não pode vencer durante o aluguel e as datas têm limites. Aqui, o `for update` trava o carro até o fim da operação, e só então o aluguel é gravado e o carro vira Alugado. Ou acontece tudo, ou nada.
>
> Além disso, este índice impede que exista mais de um aluguel em andamento para o mesmo carro. E as permissões também estão no banco: cada tabela tem regras que só deixam ver ou alterar o que a pessoa tem permissão.

Se perguntarem "por que assim?"

- *Por que a regra está no banco e não em cada aplicativo?* Porque são três aplicativos. Com a regra num lugar só, eles não conseguem se comportar diferente, e uma correção vale para os três de uma vez.
- *Por que uma função, e não dois comandos (gravar o aluguel e mudar o carro)?* Se a internet cair entre os dois, fica um aluguel com o carro "Disponível". A função é uma transação: ou grava tudo, ou nada.
- *Para que o `for update` e o índice?* Para quando duas pessoas tentam alugar o mesmo carro no mesmo segundo, por exemplo uma no celular e outra no desktop. A segunda espera a primeira terminar e recebe "não disponível".
- *Por que checar permissão no banco se a tela já esconde o botão?* Esconder o botão é conforto. Quem chamasse o banco direto passaria. A regra no banco (RLS) é a segurança de verdade.

Bloco 7 — A web (Gabriel, 1:00)

Abrir no VS Code

1. `web/includes/auth.php`, função `entrar_no_sistema` (linhas 226 a 275). Selecionar o bloqueio (linha 230), a mensagem "E-mail ou senha incorretos" (linha 253) e o `session_regenerate_id` (linha 275).
2. `web/includes/funcoes.php`, função `e()` (linha 29) e a proteção CSRF (linhas 47 a 72).

Fala

> Na web, mostro a parte de segurança. O login primeiro pergunta ao banco se o e-mail está bloqueado por excesso de tentativas. Se a senha estiver errada, a mensagem é sempre a mesma, "e-mail ou senha incorretos", para ninguém descobrir quais e-mails existem. Depois do login, a sessão ganha um número novo, para evitar roubo de sessão.
>
> Toda informação do banco que aparece na tela passa por esta função `e()`, que impede alguém de esconder um código malicioso, por exemplo no nome de um cliente. E cada formulário leva um código secreto da sessão, para outro site não enviar formulários no lugar do usuário.

Se perguntarem "por que assim?"

- *Por que a mesma mensagem para e-mail inexistente e senha errada?* Mensagens diferentes contariam a um invasor quais e-mails estão cadastrados.
- *Por que o bloqueio fica no banco e não na sessão do PHP?* Para valer nas três plataformas: errar a senha cinco vezes na web também bloqueia o celular e o desktop.
- *O que é XSS e CSRF?* XSS é alguém salvar um código (como um `<script>`) num campo para ele rodar na tela de outra pessoa; a `e()` mostra como texto. CSRF é outro site enviar um formulário usando a sessão aberta do usuário; o código secreto do formulário barra isso.
- *Por que um cliente PHP próprio para o Supabase?* Não há biblioteca oficial do Supabase para PHP. O nosso faz as requisições e renova sozinho o acesso, que dura uma hora (`supabase.php`, linha 268).

Bloco 8 — O celular (Evaldo, 1:00)

Abrir no VS Code

1. `mobile/lib/pages/locacao_page.dart`, linhas 139 a 161: a assinatura vem antes de gravar.
2. `mobile/lib/controllers/locacao_controller.dart`, linhas 35 a 55: primeiro `enviarAssinatura`, depois `registrarAluguel` com o caminho da imagem.
3. `mobile/lib/services/supabase_service.dart`, linha 400 (`enviarAssinatura`) e linha 632 (`.order('mes', ascending: true)`).

Fala

> No aplicativo, o aluguel só é gravado depois da assinatura. Se o cliente desistir de assinar, nada é gravado. A assinatura vira uma imagem PNG, que vai para uma pasta privada na nuvem, organizada por ano e mês. Só depois o app chama a mesma função do banco que o Caio mostrou, passando o caminho da imagem. Por isso a web e o desktop conseguem mostrar a assinatura.
>
> E uma lição que aprendemos: nesta biblioteca, ordenar sem dizer a direção ordena do maior para o menor. O painel chegou a mostrar o faturamento de onze meses atrás como se fosse o do mês. Corrigimos dizendo `ascending: true` em todas as listas.

Se perguntarem "por que assim?"

- *Por que guardar a assinatura como imagem num arquivo, e não dentro do banco?* Imagem é pesada. No banco fica só o caminho, e o arquivo fica no Storage, numa pasta privada que só quem tem permissão de ver locações consegue abrir.
- *Por que enviar a assinatura antes de registrar o aluguel?* Para o aluguel já nascer com o contrato assinado. Se o envio falhar, o aluguel nem é criado.
- *Por que Flutter?* Um código só gera o app para Android e iPhone, e roda também no navegador para testar.

Bloco 9 — O desktop (Caio, 1:00)

Abrir no VS Code

1. `desktop/src/main/index.js`, linhas 51 e 52: `contextIsolation: true` e `nodeIntegration: false`.
2. `desktop/src/preload/preload.js`, linha 3 (`exposeInMainWorld`) e linha 38 (`registrarAluguel`).
3. `desktop/src/main/ipc/locacoes.js`, linhas 102 a 140: recebe o pedido e chama `supabase.rpc("registrar_aluguel")`.
4. `desktop/src/main/ipc/contratos.js`, linhas 29 a 60: baixa a assinatura do celular para mostrar no contrato.

Fala

> O Electron tem duas partes: a tela, que é uma página HTML, e o processo principal, que fala com o banco. Por segurança, a tela não acessa nada do computador. Ela só pode usar os comandos que este arquivo, o preload, libera. Quando eu clico em registrar aluguel, a tela pede pelo comando `registrarAluguel`, o processo principal recebe e chama a mesma função do banco que o celular e a web usam.
>
> E é o processo principal que baixa a assinatura da pasta privada, para ela aparecer no contrato e sair no PDF.

Se perguntarem "por que assim?"

- *Por que a tela não fala direto com o banco?* Com o `contextIsolation` e o preload, mesmo que alguém conseguisse rodar um código na tela, ele só teria acesso aos comandos que liberamos, e não ao computador inteiro.
- *Por que baixar a assinatura no processo principal, e não usar um link?* A pasta é privada. Baixando com o login de quem tem permissão, a imagem chega pronta para a tela e para o PDF, sem depender de um link que expira.
- *Por que Electron?* É HTML, CSS e JavaScript, o mesmo que a web usa, e gera um programa instalável para Windows, Mac e Linux.

Encerramento

Bloco 10 — Encerramento (Evaldo, 0:30)

Na tela

1. As três plataformas abertas lado a lado (Windows: arrastar cada janela para um canto), todas mostrando a Frota.

Fala

> Foi isso o Royale Cars: um só sistema, três plataformas e um único banco de dados, com as regras e a segurança no banco, para as três funcionarem igual. Aprendemos a dividir o trabalho em partes pequenas e testadas, a usar o GitHub em equipe e que pensar no banco de dados primeiro facilita todo o resto. Obrigado por assistir!

Perguntas que o professor pode fazer

As perguntas sobre cada trecho de código estão nos blocos 6 a 9. Estas são as perguntas gerais. Combinem quem responde cada uma e treinem em voz alta.

Pergunta	Quem responde	Resposta curta
Por que Supabase e não MySQL?	Caio	É um PostgreSQL na nuvem: as três plataformas acessam pela internet, sem servidor nosso. E já traz login, armazenamento de arquivos e regras de acesso por usuário. Um MySQL local só funcionaria num computador.
Por que três plataformas?	Gabriel	Cada uma serve a um momento: a web no escritório, sem instalar nada; o desktop no balcão e na gerência; o celular no atendimento, junto do cliente, onde ele assina.
Como as senhas são guardadas?	Caio	Quem guarda é o Supabase Auth. A senha nunca fica nas nossas tabelas, só um resumo criptográfico (hash) que não dá para desfazer.
E se a internet cair no meio de um aluguel?	Evaldo	O sistema avisa para verificar a internet. Como o aluguel é uma transação, nada fica gravado pela metade.
Como os números batem nas três?	Gabriel	Os totais vêm de visões do banco, como a `vw_faturamento_mensal`: a mesma conta para as três. Foi comparando as três que achamos o bug da ordem invertida no celular.
Por que a devolução adiantada cobra só os dias usados?	Caio	Decisão do grupo: é o justo para o cliente, e a regra fica na função `registrar_devolucao`, igual nas três.
E os dados pessoais (CPF, CNH)? Pensaram na LGPD?	Gabriel	Só vê quem tem permissão, a assinatura fica numa pasta privada e toda alteração fica no histórico de ações, com quem fez e quando. Cliente sai do sistema por arquivamento, sem perder o histórico.

Pergunta	Quem responde	Resposta curta
Como foi o trabalho em equipe?	Evaldo	Cada um ficou com uma plataforma e todos ajudaram nas outras. Usamos um repositório no GitHub, com commits pequenos e testados, cada um com o nome de quem fez aquela parte.
O que mudou desde a primeira versão?	Caio	Cada um tinha feito a sua sozinho, com bancos diferentes. Em outubro unificamos: um banco, as mesmas regras, as mesmas permissões e os mesmos nomes de tela.
Vocês usaram inteligência artificial?	Quem for perguntado	Digam a verdade: usamos como assistente e professor particular, para entender, revisar e testar. Por isso cada um precisa saber explicar a sua parte, e este roteiro existe para isso.

Gravar e montar

Na hora de gravar

- Leiam a fala duas vezes antes e depois gravem falando com as palavras de vocês. Lido, soa robótico.
- Mouse calmo: parem um segundo em cima do botão antes de clicar, para quem assiste acompanhar.
- Comecem e terminem cada bloco com 2 segundos de silêncio. Isso facilita o corte.
- Errou? Parem, respirem e repitam a frase. Na edição dá para cortar o erro, sem regravar o bloco inteiro.

Nome dos arquivos (para a ordem não se perder)

`B01_abertura_caio.mp4`, `B02_web_gabriel.mp4`, `B03_celular_evaldo.mp4`, `B04_desktop_caio.mp4`, `B05_web_gabriel.mp4`, `B06_banco_caio.mp4`, `B07_codigo_web_gabriel.mp4`, `B08_codigo_celular_evaldo.mp4`, `B09_codigo_desktop_caio.mp4`, `B10_encerramento_evaldo.mp4`

Montagem (Clipchamp, que já vem no Windows 11, ou CapCut)

1. Colocar os 10 arquivos na linha do tempo, em ordem.
2. Cortar os silêncios do começo e do fim e os trechos com erro.

3. Nos blocos de código, dar zoom no trecho selecionado, para o código ficar legível.
4. Um título curto no começo de cada bloco ("Web — Gabriel", "Código do banco — Caio"). Música, se tiver, bem baixa.
5. Exportar em 1080p, MP4, e conferir que o vídeo tem no máximo 15 minutos.
6. Os três assistem ao vídeo inteiro antes de enviar.